

MAT-MEK 4270 Mandatory 1 Eirik Brenner Marthins

October 7, 2026

1.1

1.1.1

For the Poisson2D solver class to work, several functions need to be implemented: Laplace, assemble, meshfunction, get_boundary_indicies, l2-error and eval. Some comments about each function:

Laplace

This returns the second order finite difference approximation of the spatial derivative. In order to solve Poisson's equation,

$$\nabla^2 u = f,$$

numerically, we need to solve for the matrix U in

$$DU + UD^T = F$$

which cannot be done trivially as U cannot be factored out. The solution is to use the vec-trick:

$$\begin{aligned} \text{vec}(DU + UD^T) &= \text{vec}(F) \\ (I \otimes D + D \otimes I)\text{vec}(U) &= \text{vec}(F) \end{aligned}$$

This function simply returns this matrix:

```
h = self.p.L / N
D = sparse.diags([1., -2., 1.], [-1, 0, 1], (N + 1, N + 1), format="lil")
I = sparse.eye(N + 1, format="lil")
A = sparse.kron(D, I) + sparse.kron(I, D)
A /= h**2
return A
```

assemble

Here we assemble the linear algebra system as a set of equations $A\vec{u} = \vec{b}$. A we can get from the laplace-function. For $\vec{b}$, we evaluate f on the mesh. But we need to take care of the boundary conditions. On the boundary, we want to have $u = u_e$ where u_e is the exact, manufactured solution. Therefore we want the matrix A to just do the identity operation on the boundary, and we want the $\vec{b}$ to hold the corresponding u_e values instead of the f values on the boundary. In total we then get

```
xij , yij = self.create_mesh(N)

A = self.laplace(N).tolil()

b = self.meshfunction(f, xij, yij)
ueij = self.meshfunction(ue, xij, yij)

boundary = self.get_boundary_indices(N)

for i in boundary:
    A[i, :] = 0
    A[i, i] = 1

b = b.ravel()
b[boundary] = ueij.ravel()[boundary]

return A.tocsr(), b
```

meshfunction

Evaluates a Sympy expression function on an input mesh and returns it as a numpy array:

```
uij=sp.lambdify((x,y),u, "numpy")
return uij(xij,yij)
```

get_boundary_indices

Here, we need to find the indices corresponding to the boundary, to easily apply boundary conditions. This requires some thought as the solution matrix U is vectorized. Instead of a matrix of size $(N + 1) \times (N + 1)$ it is a column vector of size $(N + 1)^2 \times 1$. Numpy uses row-major vectorization. This means that the first row has indices $0, 1, \dots, N$ and the last row $N(N + 1), \dots, (N + 1)^2 - 1$. The first and last columns have to step by size $N + 1$ because of the row-major style. In total, we get the following:

```
n = N + 1
```

```

boundary = []

boundary.extend(range(n))           # top row
boundary.extend(range(N*n, n*n))   # bottom row
boundary.extend(range(0, n*n, n))   # left column
boundary.extend(range(N, n*n, n))   # right column

return np.unique(boundary)

```

l2-error

Here we calculate the l^2 error norm:

$$\sqrt{h^2 \sum_{i,j} (u_{i,j} - u_{i,j}^e)^2}$$

The implementation:

```

N = u.shape[0] - 1
xij, yij = self.create_mesh(N)

ueij = self.meshfunction(ue, xij, yij)

h = self.p.L / N
return np.sqrt(h**2 * np.sum((ueij - u)**2))

```

eval

This function is to be able to evaluate our solution u between mesh points, using interpolation. We will make this simple and use a built-in interpolation function from SciPy:

```
from scipy.interpolate import RegularGridInterpolator
```

This uses a 2D linear interpolation by default.

Implementation:

```

N = U.shape[0] - 1
mesh = self.p.create_mesh(N)

interpolator = RegularGridInterpolator((mesh, mesh), U)

return interpolator((x, y))

```

After implementation of these functions, it passes the convergence and interpolation tests.

1.2

1.2.1

Here we want to make a solver for the wave equation

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u$$

with Dirichlet boundary conditions (the function value is given on the boundary). The implementation is actually simpler than the Poisson solver, because of the hyperbolic nature of the problem. We have an initial condition, and we can then simply step the solution forward in time using the wave equation instead of solving a linear algebra system. We therefore don't need to use vectorization and the Kronecker product here.

We will use the exact solution to get boundary conditions and initial conditions:

$$u_e(t, x, y) = \sin(k_x x) \sin(k_y y) \cos(\omega t)$$

Initial conditions are thus

$$\begin{aligned} u(0, x, y) &= \sin(k_x x) \sin(k_y y) \\ \frac{\partial u(0, x, y)}{\partial t} &= -\omega \sin(k_x x) \sin(k_y y) \sin(\omega t)|_{t=0} = 0 \end{aligned}$$

Boundary conditions:

$$\begin{aligned} u(t, 0, y) &= \sin(k_x \cdot 0) \sin(k_y y) \cos(\omega t) = 0 \\ u(t, 1, y) &= \sin(k_x \cdot 1) \sin(k_y y) \cos(\omega t) = 0 \\ u(t, x, 0) &= \sin(k_x x) \sin(k_y \cdot 0) \cos(\omega t) = 0 \\ u(t, x, 1) &= \sin(k_x x) \sin(k_y \cdot 1) \cos(\omega t) = 0 \end{aligned}$$

as $\sin(k) = \sin(m\pi) = 0$. The solution is thus 0 over the complete boundary.

We can calculate U^1 from U^0 and the fact that the initial time derivative is zero everywhere. Then we can step the solution forward in time using the two step recursion from using finite differences for the second derivatives in both time and space:

$$U^{n+1} = 2U^n - U^{n-1} + (c\Delta t)^2 (D^{(2)} U^n + U^n (D^{(2)})^T)$$

And for each step apply the boundary conditions. This is simpler than for the Poisson solver as the solution is stored as a matrix and not a vector:

$$\begin{aligned} u[:, 0] &= 0 \\ u[:, -1] &= 0 \\ u[0, :] &= 0 \\ u[-1, :] &= 0 \end{aligned}$$

Implementation of the timestepping algorithm:

```

D2=self.D2(N)

data=\{\}

U0,U1=self.initialize(N,mx,my)

data[0]=U0


if store_data==1:
    data[1]=U1
elif store_data==-1:
    err=[]
    err.append(self.l2_error(U0,0))
    err.append(self.l2_error(U1,self.dt))


for n in range(2,Nt+1):
    U2=2*U1-U0+(c*self.dt)**2*(D2@U1+U1@D2.T)
    self.apply_bcs(U2)
    if store_data==-1:
        err.append(self.l2_error(U2,n*self.dt))
    U0=U1
    U1=U2
    if store_data>0 and n%store_data==0:
        data[n]=U2


if store_data>0:
    return data
elif store_data==-1:
    return 1/N, err

```

The solution is verified using the l^2 -error norm, just as for the Poisson-solver. We need to calculate the dispersion coefficient ω and the timestep Δt in two property functions. Δt is easy:

$$C = \frac{c\Delta t}{h} \Rightarrow \Delta t = \frac{Ch}{c} = \frac{C}{cN}$$

To find the dispersion coefficient we need to substitute the exact solution into

the wave equation.

$$\begin{aligned}
\frac{\partial^2 u}{\partial x^2} &= -k_x^2 u \\
\frac{\partial^2 u}{\partial y^2} &= -k_y^2 u \\
\frac{\partial^2 u}{\partial t^2} &= -\omega^2 u \\
\frac{\partial^2 u}{\partial t^2} &= c^2 \nabla^2 u = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \\
-\omega^2 u &= c^2 (-k_x^2 u - k_y^2 u) \\
\omega &= c \sqrt{k_x^2 + k_y^2} = c \pi \sqrt{m_x^2 + m_y^2}
\end{aligned}$$

1.2.2

To solve the wave equation with Neumann boundary conditions, we can inherit much from the Dirichlet solver. Only some functions need some changes: The differentiation matrix, the exact solution and the apply_boundaryconditions. This last function is very simple. As Neumann boundary conditions give free function values at the boundary (the derivative is specified instead), the boundary conditions function need not to do anything. For the differentiation matrix, we need to change some few elements. Because the derivative is zero in the direction normal to the boundary, the first and last node have different time evolution formulas. We use the derivative to calculate the value of hypothetical points just outside the boundary. The needed change of the matrix is

$$\begin{aligned}
D2[0, 1] &= 2 \\
D2[-1, -2] &= 2
\end{aligned}$$

We should check that the given exact solution actually satisfies the Neumann

boundary conditions.

$$\begin{aligned}
u_e(t, x, y) &= \cos(k_x x) \cos(k_y y) \cos(\omega t) \\
\frac{\partial u}{\partial x} &= -k_x \sin(k_x x) \cos(k_y y) \cos(\omega t) \\
\frac{\partial u}{\partial y} &= -k_y \cos(k_x x) \sin(k_y y) \cos(\omega t) \\
\frac{\partial u}{\partial x}(t, 0, y) &= -k_x \sin(k_x \cdot 0) \cos(k_y y) \cos(\omega t) = 0 \\
\frac{\partial u}{\partial y}(t, x, 0) &= -k_y \cos(k_x x) \sin(k_y \cdot 0) \cos(\omega t) = 0 \\
\frac{\partial u}{\partial x}(t, 1, y) &= -k_x \sin(k_x \cdot 1) \cos(k_y y) \cos(\omega t) = 0 \\
\frac{\partial u}{\partial y}(t, x, 1) &= -k_y \cos(k_x x) \sin(k_y \cdot 1) \cos(\omega t) = 0
\end{aligned}$$

So the Neumann boundary conditions are satisfied. Using the explained implementation, the convergence test passes.

1.2.3

Here we want to show that

$$u(t, x, y) = e^{i(k_x x + k_y y - \omega t)}$$

Satisfies the wave equation. We get

$$\begin{aligned}
\frac{\partial^2 u}{\partial x^2} &= (ik_x)^2 u \\
\frac{\partial^2 u}{\partial y^2} &= (ik_y)^2 u \\
\frac{\partial^2 u}{\partial t^2} &= (-i\omega)^2 u \\
\frac{\partial^2 u}{\partial t^2} &= c^2 \nabla^2 u = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \\
-\omega^2 u &= c^2 (-k_x^2 u - k_y^2 u) \\
\omega &= c \sqrt{k_x^2 + k_y^2} = c \pi \sqrt{m_x^2 + m_y^2}
\end{aligned}$$

So it satisfies the wave equation for the correct dispersion coefficient ω .

1.2.4

We will now substitute

$$u_{i,j}^n = e^{i(kh(i+j) - \tilde{\omega} n \Delta t)}$$

into the discretized wave equation (1.3):

$$\begin{aligned}\frac{u_{i,j}^{n+1} - 2u_{i,j}^n + u_{i,j}^{n-1}}{\Delta t^2} &= c^2 \frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n + u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{h^2} \\ u_{i,j}^n e^{-i\tilde{\omega}\Delta t} - 2u_{i,j}^n + u_{i,j}^n e^{i\tilde{\omega}\Delta t} &= \frac{c^2 \Delta t^2}{h^2} (u_{i,j}^n e^{ikh} - 2u_{i,j}^n + u_{i,j}^n e^{-ikh} + u_{i,j}^n e^{ikh} - 2u_{i,j}^n + u_{i,j}^n e^{-ikh}) \\ 2\cos(\tilde{\omega}\Delta t) - 2 &= C^2(4\cos(kh) - 4)\end{aligned}$$

Using $C = 1/\sqrt{2}$ then gives

$$\begin{aligned}2\cos(\tilde{\omega}\Delta t) - 2 &= 2\cos(kh) - 2 \\ \cos(\tilde{\omega}\Delta t) &= \cos(kh) \\ \tilde{\omega}\Delta t &= \pm kh + 2\pi N, N \in \mathbb{Z}\end{aligned}$$

Assuming the simplest solution gives

$$\begin{aligned}\tilde{\omega}\Delta t &= kh \\ \tilde{\omega} &= \frac{kh}{\Delta t} \\ \tilde{\omega} &= \frac{kh}{\Delta t} = ck \frac{h}{c\Delta t} = \frac{ck}{C} \\ \tilde{\omega} &= ck\sqrt{2} = c\sqrt{k^2 + k^2} = \omega\end{aligned}$$

So when $k_x = k_y$ and $C = 1/\sqrt{2}$, the numerical method gives an exact numerical dispersion coefficient $\tilde{\omega} = \omega$.

1.2.5

We now want to verify the result from the previous exercise, that these chosen parameters gives errors all the way down to machine precision instead of the usual $O(h^2)$. This is done simply by creating an instance of the class and solving it with `store_data=-1`. Then we get a list of the l^2 -error for each timestep. We want to check that the maximum of this error for all timesteps is never larger than 10^{-12} :

```
def test_exact_wave2d():
    sol=Wave2D()
    _, err=sol(N=100,Nt=100,mx=2,my=2,cfl=1/np.sqrt(2),store_data=-1)
    assert max(err)<1e-12, max(err)

    sol=Wave2D_Neumann()
    _, err=sol(N=100,Nt=100,mx=2,my=2,cfl=1/np.sqrt(2),store_data=-1)
    assert max(err)<1e-12, max(err)
```

1.2.6

Here we want to create a movie of the time evolution of the wave with Neumann boundary conditions. For simplicity we just reuse the code from Lecture 7:

```
import matplotlib.pyplot as plt
    import matplotlib.animation as animation

fig , ax = plt.subplots(subplot_kw={"projection": "3d"})
frames = []

sol = Wave2D_Neumann()
xij , yij=sol.create_mesh(100)
data=sol(N=100,Nt=100,cfl=1/np.sqrt(2),mx=2,my=2, store_data=5)

for n, val in data.items():
    frame = ax.plot_wireframe(xij , yij , val , rstride=2, cstride=2)

    frames.append([frame])

ani = animation.ArtistAnimation(fig , frames , interval=400, blit=True,
                                repeat_delay=1000)

ani.save('neumannwave.gif' , writer='pillow' , fps=5)
plt.show()
```